



CHEATSHEET

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

Diagnostic Scripts Cheatsheet

Quick-reference cheatsheet of diagnostic scripts and one-liners for rapid VCF component troubleshooting and health validation.

SCRIPTS

ONE-LINERS

DIAGNOSTICS

RAPID TRIAGE

VCF 9.0

VMware Cloud Foundation

VCF 9.0.1 Diagnostic Scripts Cheat Sheet

Table of Contents

1. [Overview](#)
 - [Connection Targets](#)
 2. [Quick Reference Card](#)
 3. [Status & Health Checks \(Read-Only\)](#)
 4. [NSX Diagnostics \(Read-Only\)](#)
 - [Diagnostic Escalation Path](#)
 5. [Credential Operations](#)
 6. [NSX Service Management](#)
 - [Service Recovery Sequence](#)
 7. [Database Fixes](#)
 - [Database Repair Quick Reference](#)
 8. [Troubleshooting Decision Tree](#)
-

Overview

Where commands run in this cheatsheet. Every script in this cheatsheet runs from:

- **Windows workstation (Python 3.x in CMD/PowerShell)** — Paramiko-based SSH client. Install with `pip install paramiko`. Scripts target ESXi hosts, vCenter, NSX, SDDC Manager, VCF Operations from the workstation as a single jump host.

The scripts use environment variables for credentials. Set them once per session (`set ESXI_PASSWORD=<pwd>` on Windows, `export ESXI_PASSWORD=<pwd>` on Bash). **Never paste a literal password into the scripts.** Replace any credential placeholder like `<your-password>` with values from your vault.

20 Python diagnostic scripts for VCF 9.0.1 nested lab troubleshooting. All use **Paramiko** for SSH and run from a Windows workstation.

```
pip install paramiko
python <script-name>.py
```

Connection Targets

Target	IP	User	Purpose
SDDC Manager	192.168.1.241	vcf	API gateway, database access (su to root)
NSX Node	192.168.1.71	root	Direct NSX service management
NSX VIP	192.168.1.70	admin	NSX cluster API (via curl from SDDC Mgr)

Note: All scripts use the standard lab password. If `vcf` is locked out: `faillock --user vcf --reset` from SDDC Manager console as root.

Quick Reference Card

Scenario	Script
Is everything healthy?	<code>python quick_status.py</code>
NSX slow after boot?	<code>python nsx_monitor.py</code>
Credential operation failed?	<code>python check_remediate_error.py</code>
Need to update NSX password?	<code>python nsx_cred_update.py</code>
NSX CPU overloaded?	<code>python nsx_slim.py</code>
Put NSX services back?	<code>python nsx_restart_all.py</code>
Clear stale DB locks?	<code>python clear_locks.py</code>
Fix stuck tasks in DB?	<code>python fix_stuck_tasks.py</code>
Full cascade fix?	<code>python full_remediate_fix.py</code>
System clean after fix?	<code>python final_check.py</code>

Status & Health Checks (Read-Only)

Script	Connects To	What It Does
<code>quick_status.py</code>	SDDC Manager	Start here. NSX status, VIP health, resource locks, notifications, credentials
<code>final_check.py</code>	SDDC Manager	Lightweight: notifications and resource locks only
<code>diag.py</code>	localhost	DNS resolution, TCP 443 connectivity, ARP/routing from Windows host
<code>nsx_monitor.py</code>	NSX Node	Polls cluster status + load avg every 60s for 10 iterations

NSX Diagnostics (Read-Only)

Script	Connects To	What It Does
<code>nsx_check.py</code>	SDDC Manager	Tests both NSX VIP and direct node connectivity — diagnoses VIP failover issues
<code>nsx_diag.py</code>	NSX Node	Top CPU consumers, disk space, service health via API, catalina errors
<code>nsx_resource_check.py</code>	SDDC Manager	NSX clusters, credentials, warnings, DB resource state
<code>sddc_nsx_status.py</code>	SDDC Manager	Compares SDDC Manager's NSX status vs actual NSX VIP cluster status

Diagnostic Escalation Path

```
python quick_status.py      # 1. Overall health
python nsx_check.py         # 2. VIP + node connectivity
python nsx_diag.py          # 3. Performance & services
python sddc_nsx_status.py   # 4. SDDC Manager vs NSX sync
```

Credential Operations

Script	Modifies	What It Does
<code>nsx_cred_update.py</code>	Yes	Full workflow: health checks, lists credentials, updates admin API, monitors 200s
<code>nsx_retry_when_ready.py</code>	Yes	Waits up to 15 min for NSX API, then submits update with 450s monitoring
<code>check_disconnected.py</code>	No	Inspects all credential objects for connection status fields
<code>check_remediate_error.py</code>	No	Failed task details with full error messages, NSX connectivity test, log search

WARNING: Do not run credential update scripts if NSX status is not **ACTIVE** in SDDC Manager or **STABLE** at the VIP. A failed update creates stale locks and stuck tasks, requiring database repair.

NSX Service Management

Script	Action	What It Does
<code>nsx_slim.py</code>	Stops	Stops 5 non-essential services to free CPU during boot storm
<code>nsx_restart_all.py</code>	Starts	Restarts all services stopped by <code>nsx_slim.py</code>
<code>nsx_fix_svc.py</code>	Restarts	Restarts search, nsx-sha, nsx-appl-proxy, validates health

Service Recovery Sequence

```
python nsx_slim.py           # Free CPU (if load > 30)
# Wait for load to drop below 15
python nsx_restart_all.py    # Bring services back
python nsx_check.py          # Verify cluster health
```

Database Fixes

Scripts that modify SDDC Manager's PostgreSQL database. All use the trust auth workaround (backup `pg_hba.conf`, set trust, fix, restore).

Script	What It Does
<code>clear_locks.py</code>	Fixes NSX status (ACTIVATING/ERROR -> ACTIVE), clears <code>lock</code> table, restarts operationsmanager
<code>fix_stuck_tasks.py</code>	Marks stuck <code>task_metadata</code> as resolved, clears <code>task_lock</code> , fixes <code>execution_to_task</code> orphans
<code>full_remediate_fix.py</code>	Complete cascade fix: NSX health check + DB fix (status + locks + tasks) + service restart
<code>find_pg_pass.py</code>	Searches for PostgreSQL password in config files (read-only)
<code>get_task.py</code>	Retrieves task details by ID with subtask errors (edit <code>task_id</code> before running)

Always use `PAGER=cat` when running `psql` to prevent pager traps in remote sessions.

Database Repair Quick Reference

```

-- Connect: su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform"

-- Fix NSX status (covers ACTIVATING and ERROR)
UPDATE nsxt SET status = 'ACTIVE' WHERE status != 'ACTIVE';

-- Clear stale locks
DELETE FROM lock;

-- Resolve stuck tasks
UPDATE task_metadata SET resolved = true WHERE resolved = false;
DELETE FROM task_lock;

```

Full procedure: [Troubleshooting Handbook](#) Section 10

Troubleshooting Decision Tree

Problem: Credential operation failed

```

|
+-- python quick_status.py
|   |
|   +-- NSX Status = ACTIVATING or ERROR?
|       |   +-- python clear_locks.py (fix DB status + locks)
|       |   +-- python fix_stuck_tasks.py (resolve stuck tasks)
|       |   +-- OR: python full_remediate_fix.py (all-in-one)
|       |
|       +-- NSX VIP returning 503?
|           |   +-- python nsx_diag.py (check load)
|           |   +-- Load > 30? -> python nsx_slim.py (free CPU)
|           |   +-- Wait -> python nsx_monitor.py (track recovery)
|           |
|           +-- All green?
|               +-- python nsx_cred_update.py (retry update)

```

Document Information

Field	Value
Document Title	VCF 9.0.1 Diagnostic Scripts Cheat Sheet
Version	2.0
Last Updated	February 2026
Script Count	20 scripts
Prerequisite	Python 3.x + Paramiko (<code>pip install paramiko</code>)
Environment	Dell Precision 7920, VMware Workstation Nested Lab

Scripts are for lab/educational purposes. Verify target hosts before running. Database-modifying scripts back up and restore `pg_hba.conf` automatically.

(c) 2026 Virtual Control LLC. All rights reserved.